

Università degli Studi di Torino

Dipartimento di informatica



Tesi di Laurea Triennale in Informatica

Overlay network per la federazione di risorse nel compute continuum

Relatore:

Prof. Paolo Castagno

Candidato:

Salvatore Francesco Indino

Anno Accademico 2022/2023

Alla mia famiglia,
quella di sangue e quella per scelta.

Abstract

Questa tesi presenta un'implementazione di un'infrastruttura di rete overlay utilizzando Nebula per abilitare la migrazione della computazione tra i nodi in un cluster Kubernetes. Nebula, una soluzione open-source per reti virtuali sicure, viene preferita rispetto a una VPN tradizionale per la sua flessibilità, scalabilità e capacità di gestire topologie di rete complesse. Attraverso l'utilizzo di Nebula, si superano le limitazioni della rete fisica sottostante, consentendo una migrazione fluida e efficiente delle risorse computazionali all'interno del cluster Kubernetes.

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata

Contents

1	Introduzione	6
2	Docker	8
2.1	Rete di Docker	9
2.2	Definizione delle immagini docker	13
3	Nebula	15
3.1	Utilizzo di Nebula	15
3.2	Implementazione di una rete Nebula	16
3.2.1	Ruolo dei nodi faro in Nebula	16
3.2.2	Ruolo dei nodi client in Nebula	17
3.2.3	Avvio della rete Nebula	17
3.2.4	Creazione della Certificate Authority	18
3.2.5	Creazione dei certificati	18
3.2.6	Configurazione dei nodi	19
3.3	Il DNS in Nebula	21
3.4	Nebula vs VPN	22
4	Kubernetes	24
4.1	Componenti Principali	24
4.1.1	Master Node	24
4.1.2	Worker Node	25
4.1.3	Kubectl	25
4.1.4	Altri componenti	26
5	K3S Lightweight Kubernetes	28
5.1	Implementazione con K3s	30
5.1.1	K3s server	30
5.1.2	K3s agent	31
6	Conclusioni	33

List of Figures

1	Architettura generale	7
2	Docker e VMs	8
3	Toplogia rete Docker	9
4	Topologia rete Nebula	16
5	Overview implementazione Kubernetes	24
6	Generco Pod Kubernetes	27
7	Generica architettura k3s	28
8	Architettura k3s	30

1 Introduzione

La crescente necessità di tecnologie con bassa latenza e elevate capacità di calcolo sta mettendo in discussione l'efficacia del tradizionale modello basato sui datacenter. Questi centri elaborano i dati e li trasmettono agli utenti finali, ma talvolta non sono ottimali, specialmente in settori come la telemedicina o la guida autonoma, dove la latenza elevata può compromettere la sicurezza e l'efficacia del servizio.

Di conseguenza si sta assistendo a una migrazione verso un modello orientato verso dispositivi più vicini all'utente, chiamati anche dispositivi periferici, attribuendo loro un ruolo attivo nella risoluzione del problema. Questi dispositivi poichè sono distribuiti più vicino agli utenti finali, consentono un'elaborazione e una risposta più rapida, riducendo così la latenza e migliorando l'esperienza utente. In questo nuovo approccio quindi, i dispositivi non solo raccolgono e trasmettono i dati, ma anche eseguono calcoli e decisioni localmente, offrendo una soluzione più efficiente e resiliente alle esigenze di applicazioni sensibili alla latenza.

Per implementare questo approccio bisogna affrontare diverse sfide, come ad esempio, trovare un modo per spostare la computazione da un dispositivo periferico ad un altro in modo affidabile senza perdere i progressi

Una possibile soluzione è quella di avere dei servizi che vengono gestiti tra dispositivi che si spostano senza supporto di una infrastruttura fissa, sfruttando la virtualizzazione ¹

Tale approccio facilita e ottimizza la gestione delle risorse, garantendo al contempo resilienza e flessibilità, come ad esempio, nel contesto delle micro-cloud² veicolari in cui i servizi e le funzioni di rete possono essere istanziati in un cluster e migrati dinamicamente in un altro sotto il coordinamento di un nodo che gestisce e monitora l'applicazione.

Un esempio pratico potrebbe essere un'applicazione in ambito automotive per la gestione di veicoli autonomi in prossimità di incroci. Quest'ultimi sono serviti da micro-cloud quindi tutte le vetture che entrano in quella zona prendono parte a questo cluster ottenendo le informazioni necessarie dalle vetture già presenti e che sono in procinto di andare via. E' proprio sul

¹Un esempio di questa soluzione è descritta nell'articolo "V-Edge: Virtual Edge Computing as an Enabler for Novel Microservices and Cooperative Computing" di Dressler, Falko and Chiasserini, Carla Fabiana and Fitzek, Frank HP and Karl, Holger and Cigno, Renato Lo and Capone, Antonio and Casetti, Claudio and Malandrino, Francesco and Mancuso, Vincenzo and Klingler, Florian and others

²Ambiente di cloud computing ottimizzato per dimensioni ridotte o per implementazioni specifiche, come ad esempio in ambienti con limitate risorse hardware o in contesti particolari come veicoli

punto della migrazione di queste informazioni che si concentrerà la presente tesi.

L'architettura sviluppata virtualizzando i servizi è composta nel seguente modo:

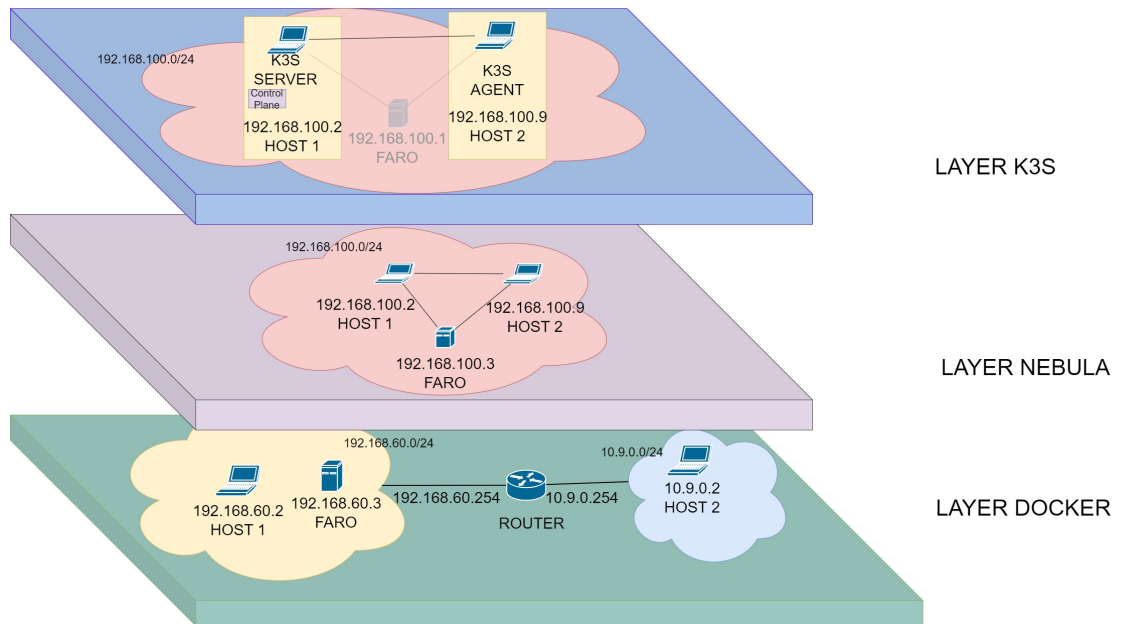


Figure 1: Architettura Generale

Si possono vedere 3 livelli, il primo è il **Layer Docker** (Capitolo 2) che, in un caso reale, sarebbe la rete fisica ma, poichè non disponiamo di una infrastruttura di rete abbiamo usato Docker per virtualizzare tale infrastruttura.

Al di sopra di questo abbiamo il **layer Nebula** (Capitolo 3) nel quale viene implementata un overlay network³ che ci permette di agire come se gli host facessero parte di una sola rete logica.

Infine, al di sopra del Layer Nebula, c'è il **Layer K3s** (Capitolo 5) nel quale è presente una distribuzione di Kubernetes che gestisce i container dove avvengono le computazioni e permette di spostare, creare ed eliminare quest'ultimi.

³Rete logica costruita su di un'altra rete

2 Docker

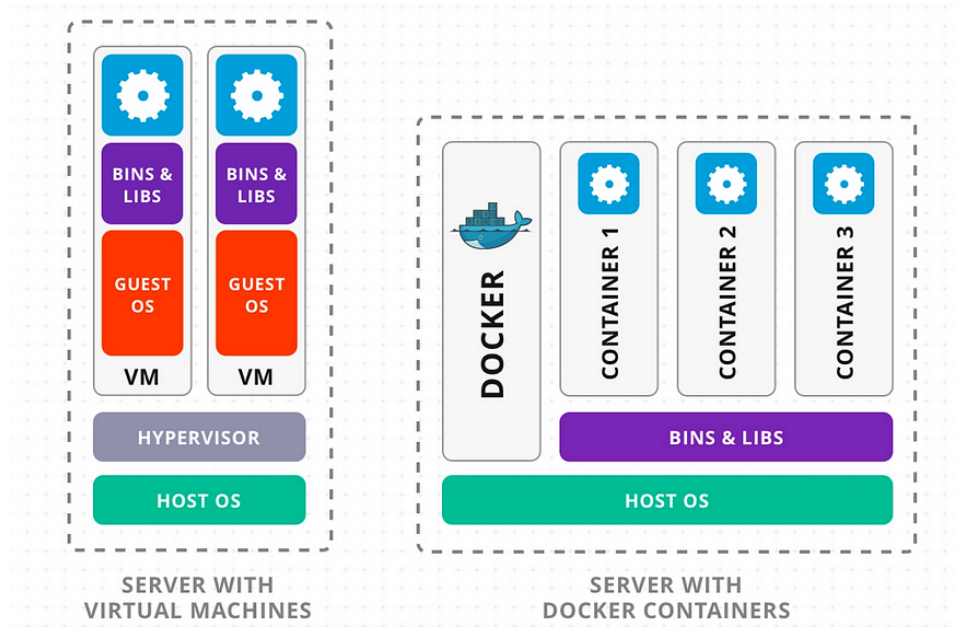


Figure 2: Docker e VMs ⁴.

Docker è una tecnologia di virtualizzazione a livello di sistema operativo che consente di creare, distribuire e gestire applicazioni all'interno di container. Un container Docker è un'istanza di un'immagine, che contiene tutto il necessario per eseguire un'applicazione, inclusi il codice, le librerie e le dipendenze, isolati dal sistema operativo host e da altri container.

Il funzionamento di Docker si basa su concetti chiave come le immagini e i container. Un'immagine Docker è un pacchetto leggero e autosufficiente che include il codice dell'applicazione, le dipendenze e le istruzioni per eseguire l'applicazione. Le immagini sono create utilizzando un file di definizione chiamato Dockerfile, che specifica i passaggi necessari per costruire l'immagine.

Al contrario di una classica macchina virtuale, che richiede la virtualizzazione di un intero sistema operativo, Docker utilizza i container, una forma leggera di virtualizzazione che condividono le risorse e il kernel del sistema

⁴Fonte: <https://hanwenzhang123.medium.com/docker-vs-virtual-machine-vs-kubernetes-overview-389db7de7618>

host utilizzando la tecnologia di namespace e di controllo dei processi per isolare l'applicazione e le sue risorse dal resto del sistema.

Una volta che un'immagine è stata creata, può essere utilizzata per avviare uno o più container Docker. Un container è un'istanza eseguibile di un'immagine Docker, che esegue un'applicazione in un ambiente isolato e autonomo.

2.1 Rete di Docker

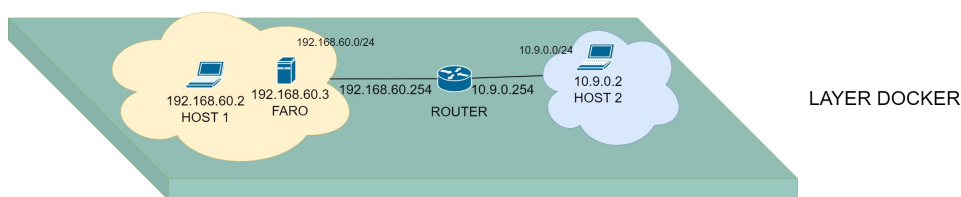


Figure 3: Topologia Della rete Docker

Il primo livello è l'implementazione della topologia di rete come illustrato nella Figura 3 che, come accennato, vuole simulare la rete fisica reale nella quale fanno parte host connessi attraverso internet, potenzialmente da diverse parti del mondo, e che vogliono comunicare. Nella suddetta topologia ci due ruoli distinti per i container: quello del **Router** e quello degli **Host**. Il container Router, è configurato per simulare, appunto, il comportamento di un router e il suo Docker Compose è definito nel seguente modo:

```
router:
  image: router
  privileged: true
  container_name: router
  cap_add:
    - ALL
  sysctls:
    - net.ipv4.ip_forward=1
  volumes:
    - ./volumes:/volumes
  networks:
    net-10.9.0.0:
      ipv4_address: 10.9.0.254
    net-192.168.60.0:
```

```
    ipv4_address: 192.168.60.254
  ports:
    - 4242
    - 4241
    - 4342
    - 80
    - 8080
    - 6444
    - 6443
    - 443
    - 5000
    - 23788
    - 2379
    - 2375
```

Il servizio "router" è stato configurato con privilegi (`privileged: true`), il che gli consente di accedere a tutte le risorse del kernel dell'host. Inoltre, sono state aggiunte tutte le capacità (`cap_add: - ALL`), estendendo ulteriormente l'accesso del servizio alle risorse di sistema.

Utilizzando il parametro `sysctls`, è stato abilitato l'instradamento IP (`net.ipv4.ip_forward=1`) all'interno del container, consentendo al container di agire come un router.

Il servizio è inoltre associato a due reti (`networks`) e sono state aperte diverse porte (`ports`) per consentire la comunicazione con altri container o host. Le reti `net-10.9.0.0` e `net-192.168.60.0` sono state definite anch'esse nel docker compose, e al container sono stati assegnati a indirizzi IP specifici all'interno di queste reti.

Questa configurazione fornisce una base solida per lo sviluppo e il testing della rete overlay Nebula, consentendoli di simulare il comportamento di un router all'interno dell'ambiente.

Per quanto riguarda gli host, sono stati configurati come segue:

```
host1:
  image: host1
  privileged: true
  container_name: host1
  cap_add:
    - ALL

  volumes:
```

```
- ./volumes:/volumes
networks:
  net-192.168.60.0:
    ipv4_address: 192.168.60.2

ports:
- 4242
- 4241
- 4342
- 80
- 8080
- 6444
- 6443
- 443
- 5000
- 23788
- 2379
- 2375
```

```
host2:
  image: host2
  privileged: true
  container_name: host2
  cap_add:
    - ALL

  volumes:
    - ./volumes:/volumes
  networks:
    net-10.9.0.0:
      ipv4_address: 10.9.0.2
  ports:
    - 4242
    - 4241
    - 4342
    - 80
    - 8080
    - 6444
    - 6443
    - 443
```

- 5000
- 23788
- 2379
- 2375

```
faro:
  image: faro
  privileged: true
  container_name: faro
  cap_add:
    - ALL
  sysctls:
    - net.ipv4.ip_forward=1
  volumes:
    - ./volumes:/volumes
  networks:
    net-192.168.60.0:
      ipv4_address: 192.168.60.3
  ports:
    - 4242
    - 4241
    - 4342
    - 80
    - 8080
    - 6444
    - 6443
    - 443
    - 5000
    - 23788
    - 2379
    - 2375
```

La configurazione degli host è stata eseguita in modo da consentire la comunicazione tra di essi all'interno della rete, anche con l'aggiunta di appropriate regole di routing.

2.2 Definizione delle immagini docker

Per definire le immagini Docker utilizzate nel progetto, si è seguito un processo che include la creazione di Dockerfiles per la costruzione di immagini Docker utilizzando il comando `docker build`. Ogni Dockerfile contiene le istruzioni necessarie per configurare l'ambiente di esecuzione del servizio specifico e per preparare il container per l'esecuzione. Per definire le immagini si è usata come base l'immagine `docker:dind`, una distribuzione di alpine linux che include il supporto per docker-in-docker il quale permette al suo interno l'esecuzione di docker, necessario a K3s per espletare le sue funzioni, le quali verranno trattate nel capitolo 4. Vista la presenza di due ruoli anche le immagini differiscono in base ad essi, di fatto le immagini degli host sono arricchite aggiungendo K3s prendendo come base il progetto `github k3s-dind`⁵ ed effettuando le opportune modifiche e adattamenti per il presente caso d'uso. Il Dockerfile del router è molto scarno in quanto il suo compito è solo quello di effettuare il forwarding del traffico ed è definito come segue:

```
#FROM alpine:latest
FROM docker:dind
LABEL maintainer="Salvatore Francesco Indino"

# Esposizione delle porte utilizzate
EXPOSE 4242/tcp
EXPOSE 4242/udp
EXPOSE 4241/tcp
EXPOSE 4241/udp
EXPOSE 4243/tcp
EXPOSE 4243/udp
EXPOSE 80
EXPOSE 8080
EXPOSE 6444
EXPOSE 6443
EXPOSE 2378
EXPOSE 32430
EXPOSE 5678
EXPOSE 8443
```

⁵realizzato da `unboundedsystems`, raggiungibile dal seguente link <https://github.com/unboundedsystems/k3s-dind>

```
# Fai rimanere il container in esecuzione senza  
# fare alcuna operazione  
CMD ["tail", "-f", "/dev/null"]
```

Più interessante è invece il Dockerfile degli host, definito come segue:

```
FROM docker:19.03.7-dind

ENV K3S_VERSION="v1.18.4%2Bk3s1"
EXPOSE 8443

ADD https://github.com/rancher/k3s/releases/download/  
    /${K3S_VERSION}/k3s /usr/local/bin/k3s
COPY kubect1 start-k3s.sh get-kubeconfig.sh /usr/  
    local/bin/

RUN apk --no-cache add bash && \  
    chmod a+x \  
        /usr/local/bin/k3s \  
        /usr/local/bin/start-k3s.sh \  
        /usr/local/bin/get-kubeconfig.sh \  
        /usr/local/bin/kubect1 \  
        && \  
    WORKDIR /nebula

COPY ./ca.key /nebula/  
COPY ./ca.crt /nebula/  
COPY ./host.crt /nebula/  
COPY ./host.key /nebula/  
COPY ./config.yaml /nebula/  
COPY ./nebula /nebula/  
COPY start_services.sh /usr/local/bin/start_services  
    .sh

# Imposta i permessi di esecuzione  
RUN chmod +x /nebula/nebula  
RUN chmod +x /usr/local/bin/start_services.sh

ENTRYPOINT [ "/usr/local/bin/start_services.sh" ]
```

3 Nebula

Nebula è una piattaforma open-source sviluppata da Slack Technologies che fornisce strumenti per la creazione, la gestione e la sicurezza di reti virtuali. Si tratta di un software progettato per semplificare la connettività e la comunicazione tra i dispositivi in un'infrastruttura IT distribuita, offrendo un'alternativa moderna e altamente sicura per la connettività di rete.

La caratteristica principale di Nebula è l'utilizzo di un approccio di rete overlay, che consente di creare una rete virtuale sopra una rete fisica esistente. Questo permette di superare le limitazioni della topologia di rete sottostante e di fornire una maggiore flessibilità nella configurazione e gestione della rete. Nebula implementa un modello di sicurezza avanzato basato su crittografia end-to-end, utilizzando certificati per l'autenticazione con crittografia basata su curve ellittiche per garantire la confidenzialità e l'integrità dei dati trasmessi sulla rete overlay.

Le potenziali applicazioni di Nebula includono il deployment di microservizi in un'architettura basata su container, l'accesso remoto sicuro a risorse di rete interne e la gestione della connettività in ambienti distribuiti e complessi. Nebula inoltre offre il servizio, seppur sperimentale, di DNS che vedremo più avanti nel Capitolo 3.3

3.1 Utilizzo di Nebula

Il software Nebula, nell'ambito del presente progetto di tesi, svolge la funzione di consolidare e integrare in una singola rete logica tutti gli host coinvolti. Questo processo consente di superare eventuali incompatibilità tra i dispositivi, semplificando così la migrazione della computazione dei cluster Kubernetes.

3.2 Implementazione di una rete Nebula

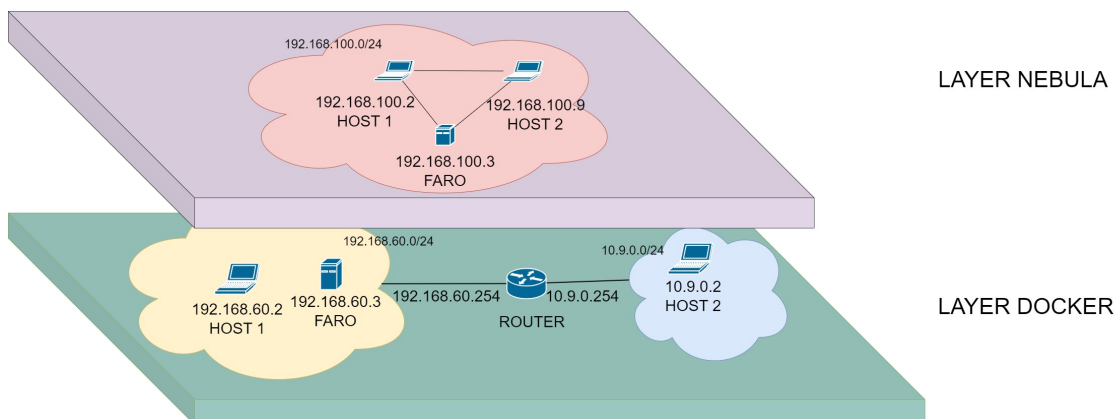


Figure 4: Topologia Rete Nebula

L'implementazione di una rete Nebula avviene attraverso diversi passaggi chiave, Creazione della Certificate Authority, Firma dei certificati, configurazione dei nodi. Prima però esaminiamo le due tipologie di nodi.

3.2.1 Ruolo dei nodi faro in Nebula

Nel contesto di Nebula, il faro svolge un ruolo cruciale in diversi momenti. Esso funge da punto di ingresso per gli host esterni e, pertanto, richiede un indirizzo IP pubblico per consentire la connettività alla rete Nebula. Questo IP è essenziale per permettere agli host esterni di stabilire una connessione con la rete Nebula e accedere alle risorse condivise all'interno di essa.

In secondo luogo, il faro ha la responsabilità di tenere traccia di tutti gli altri host presenti sulla rete Nebula. Agisce come un registro centrale degli host, memorizzando informazioni quali gli indirizzi IP, i nomi presenti nei certificati e altre informazioni di identificazione. Questo registro permette agli host di individuarsi reciprocamente all'interno della rete, facilitando la comunicazione e la condivisione delle risorse.

Infine, il faro può assumere anche la funzione di server DNS all'interno della rete Nebula. Questo significa che è in grado di risolvere i nomi degli host in indirizzi IP all'interno della rete, consentendo agli host di comunicare tra loro utilizzando nomi di dominio anziché indirizzi IP. Questa funzionalità semplifica la gestione della rete e migliora l'usabilità, consentendo agli utenti di identificare e raggiungere gli host sulla rete Nebula in modo intuitivo e

conveniente.

3.2.2 Ruolo dei nodi client in Nebula

I nodi client, partecipano anche loro in modo attivo per il mantenimento della rete con diverse responsabilità tra cui:

- **Routing peer-to-peer:** Ogni nodo contribuisce alla gestione del traffico di rete, decidendo quali percorsi sono ottimali per instradare i dati. Questo coinvolgimento attivo è fondamentale per garantire che il traffico venga indirizzato in modo efficiente attraverso la rete, minimizzando ritardi e congestioni.
- **Resilienza:** I nodi sono in grado di adattarsi alle variazioni nella topologia di rete e di ripristinare rapidamente le connessioni in caso di guasti o interruzioni. Questa capacità di adattamento attivo contribuisce alla resilienza complessiva della rete Nebula, garantendo che possa continuare a funzionare in modo affidabile anche in condizioni avverse.
- **Sicurezza:** I nodi collaborano per proteggere la rete Nebula da minacce esterne e interne. Questo coinvolgimento attivo include la partecipazione alla rilevazione delle intrusioni, alla convalida delle transazioni e all'implementazione di protocolli di sicurezza per proteggere l'integrità dei dati e garantire la privacy degli utenti.

3.2.3 Avvio della rete Nebula

L'aggiunta di un nodo client Nebula segue una serie di fasi, che includono l'identificazione dei nodi faro disponibili, l'avvio di un handshake sicuro e l'autenticazione della connessione. Qui di seguito, assumendo la presenza di almeno un nodo faro attivo, esamineremo il procedimento attraverso il quale un nodo client si connette alla rete Nebula:

- **Inizializzazione del nodo non Faro:** Per avviare la connessione, l'host non faro deve essere inizializzato e configurato correttamente. Ciò può includere la generazione di una chiave crittografica pubblica/privata e l'assegnazione di un identificatore univoco all'host.
- **Ricerca dei Nodi Faro:** L'host non faro cerca nella rete Nebula i nodi faro disponibili. Questi sono punti di accesso privilegiati che consentono agli host non faro di connettersi alla rete.

- **Handshake con i Nodi Faro:** Una volta individuati i nodi faro, l'host non faro avvia il processo di handshake con uno o più di essi. Durante il handshake, l'host invia una richiesta di connessione contenente la propria chiave pubblica e altre informazioni di identificazione.
- **Autenticazione e Accettazione della Connessione:** I nodi faro ricevono la richiesta di connessione e autenticano l'host non faro. Questo può coinvolgere la verifica dei certificati dell'host e altre procedure di sicurezza. Se l'autenticazione ha successo, il nodo faro accetta la connessione con l'host non faro.
- **Instradamento del Traffico:** Una volta che l'host non faro è connesso a uno o più nodi faro, può iniziare a instradare il traffico attraverso la rete Nebula. I nodi faro agiscono come ponti per consentire all'host di comunicare con altri nodi nella rete.
- **Partecipazione alla Rete:** Ora che è connesso alla rete Nebula, l'host non faro può partecipare alle attività di rete, come lo scambio di dati, la convalida delle transazioni o l'accesso a servizi e risorse disponibili sulla rete.

Nelle prossime sottosezioni si vedrà in dettaglio parte del processo di configurazione dei nodi, e il processo di creazione della rete Nebula.

3.2.4 Creazione della Certificate Authority

Per prima cosa si procede con la creazione della Certificate Authority (CA) il quale, nella sua forma più semplice, è composta da due file, un certificato CA e una chiave privata associata. Un certificato CA viene distribuito e considerato attendibile da ogni host della rete. La chiave privata della CA non deve essere distribuita e può essere mantenuta offline quando non viene utilizzata per aggiungere host a una rete Nebula. La creazione della suddetta CA avviene con il seguente comando.

```
./nebula-cert ca -name "Tesi"
```

3.2.5 Creazione dei certificati

I certificati consentono l'autenticazione degli host e l'individuazione univoca all'interno della rete. Durante questo processo, vengono specificati l'indirizzo IP e il gruppo di appartenenza di ciascun host, permettendo una corretta configurazione e gestione della rete Nebula.

Questi certificati sono fondamentali per garantire la sicurezza e l'autenticità delle comunicazioni sulla rete Nebula, consentendo solo agli host autorizzati di partecipare alla rete e di comunicare tra loro in modo sicuro. La creazione e firma dei certificati avviene con il seguente comando

```
./nebula-cert sign -name "faro" -ip "
192.168.100.1/24"
./nebula-cert sign -name "host1" -ip "
192.168.100.2/24" -groups "host"
./nebula-cert sign -name "host2" -ip "
192.168.100.9/24" -groups "host"
```

Una volta generati i certificati, è possibile procedere con la configurazione dei nodi della rete Nebula. Ogni nodo deve essere correttamente configurato con il proprio certificato e le informazioni relative all'indirizzo IP e al gruppo di appartenenza. Questa configurazione consente ai nodi di autenticarsi alla rete e di partecipare alle comunicazioni all'interno del gruppo specificato.

3.2.6 Configurazione dei nodi

La configurazione dei nodi nebula avviene attraverso un file chiamato `config.yaml` diverso da nodo in nodo, in particolar modo se il nodo in questione è un faro. Le parti interessanti sono della configurazione del nodo faro sono le seguenti:

- Specifica dei certificati

```
pki:
  ca:    ./ca.crt    #Certificate Authority (CA)
  cert:  ./host.crt  #Certificato del nodo firmato
                  dalla CA
  key:   ./host.key  #Chiave privata del nodo
```

- Specifica del tipo di nodo

```
lighthouse:
  am_lighthouse: true    #Specifica che il nodo e
    ' un faro
  serve_dns: true        #Abilita il servizio DNS
  dns:
    host: 0.0.0.0        #Ip su cui ascolta il DNS
    port: 53             #Porta su cui ascolta il
      DNS
```

- Setting del Firewall

```
firewall:
conntrack:
  tcp_timeout: 12m
  udp_timeout: 3m
  default_timeout: 10m

outbound:                #Traffico in Uscita
- port: any
  proto: any
  host: any

inbound:                 #Traffico in Entrata
- port: any
  proto: any
  host: any

- port: 8443
  proto: any
  host: any
```

I nodi client conservano la specifica dei certificati e il setting del firewall in modo pressochè identico, mentre la specifica del tipo di nodo è, ovviamente, differente.

```
static_host_map:

"192.168.100.1": ["192.168.60.3:4242"]
# Viene specificato l'ip nebula e quello
  routable

# "{nebula ip}":
  ["{routable ip/dns name}:{routable port}
  "]

lighthouse:

  am_lighthouse: false #Specifica che il nodo
    NON e' un faro

  serve_dns: false
  dns:
    host: 0.0.0.0
    port: 53
    nameserver: 192.168.100.1
  interval: 60

  hosts:
    - "192.168.100.1" #Inserisce nella route
      table l'IP nebula del faro
```

Ora che si hanno i nodi configurati si può far partire la rete Nebula eseguendo il seguente comando su ogni nodo, specificando il file di configurazione con il flag `-config`:

```
./nebula -config /nebula/config.yaml
```

3.3 Il DNS in Nebula

Come detto precedente, Nebula include un server DNS sperimentale che fornisce la risoluzione dei nomi di dominio all'interno della rete Nebula. Tuttavia, è importante notare che questo servizio presenta alcune limitazioni dovute alla sua natura sperimentale:

- Il server DNS integrato risponderà solo alle query per host Nebula conosciuti e non effettuerà query upstream a un server DNS secondario. Questo significa che il server DNS Nebula fornirà solo risposte per gli host all'interno della rete Nebula e non per risorse esterne alla rete.
- Il nome host di un host è limitato al nome nel suo certificato Nebula. Questo significa che la risoluzione dei nomi avviene in base al nome specificato nel certificato Nebula di ciascun host.
- Non è possibile registrare nomi host aggiuntivi, come CNAME aggiuntivi per un determinato host Nebula o record A aggiuntivi per la gestione di host non Nebula. Il server DNS Nebula fornisce solo la risoluzione dei nomi basata sui certificati Nebula e non supporta la registrazione di record DNS personalizzati.
- I nomi duplicati nei certificati Nebula possono causare risposte non stabili dal server DNS. È importante evitare duplicati nei nomi host nei certificati Nebula per garantire un funzionamento corretto del server DNS Nebula.
- Se il nome nel certificato Nebula non è un nome host valido, il server DNS Nebula restituirà un risultato vuoto. È fondamentale che i nomi host nei certificati Nebula siano correttamente configurati per garantire una risoluzione dei nomi affidabile. [1]

Queste limitazioni evidenziano il fatto potrebbe non essere adatto a tutte le esigenze di risoluzione dei nomi all'interno della rete.

3.4 Nebula vs VPN

Una configurazione VPN tradizionale si basa su un server VPN con un indirizzo IP pubblico, il quale può essere situato sul firewall/router periferico o come server separato all'interno della rete privata. Questo server funge da punto di accesso centrale per i nodi client, i quali si connettono ad esso da reti non attendibili, come Internet, per accedere alle risorse all'interno della rete privata. Il server VPN gestisce l'autenticazione e la crittografia delle comunicazioni tra i nodi client e la rete privata, creando un tunnel sicuro attraverso il quale i dati possono viaggiare in modo protetto. Questo approccio consente agli utenti remoti di accedere in modo sicuro alle risorse interne della rete, come file, stampanti o applicazioni, come se fossero fisicamente connessi alla stessa rete locale. Al contrario in una rete overlay, come Nebula, adottano un approccio radicalmente diverso rispetto alle VPN tradizionali

creando una rete virtuale sovrapposta a quella fisica, con indirizzi IP e nomi host statici, nella quale i nodi che ne fanno parte agiscono come se fossero nella stessa rete locale. Infatti il nodo visibile dall'esterno detto faro ha il solo compito di autenticare i vari nodi alla rete i quali poi saranno connessi con una rete mesh crittografata end-to-end sicura tra i dispositivi. In questo modello, ogni dispositivo è essenzialmente un nodo della rete e può comunicare direttamente con gli altri nodi, eliminando la dipendenza da un singolo punto di controllo. Questa architettura decentralizzata offre maggiore flessibilità, resistenza agli attacchi e facilità di scalabilità rispetto alle VPN tradizionali. Inoltre, riduce la necessità di configurare e gestire un server centrale, semplificando notevolmente il processo di implementazione e manutenzione della rete. Una differenza sostanziale è l'accesso alle risorse locali quali, stampanti di rete, dispositivi NAS, etc. Infatti mentre con la VPN questo accesso è integrato, un approccio alternativo il viene fornito nella documentazione ufficiale, che consiste nell'estendere la rete oltre agli host overlay.

4 Kubernetes

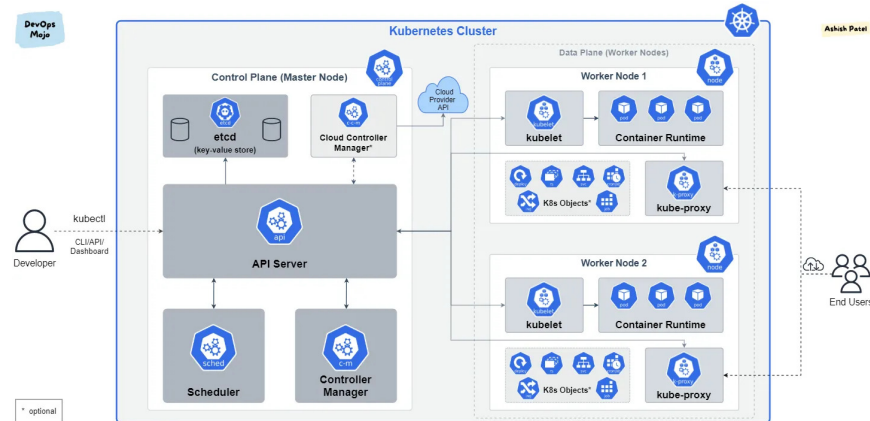


Figure 5: Overview di una implementazione generica di Kubernetes⁶.

Kubernetes è un sistema open-source sviluppato da Google per automatizzare la gestione, il deployment e la scalabilità di applicazioni contenute in un container Docker. La sua architettura è basata su un modello client-server e consiste in diversi componenti che collaborano per fornire un ambiente di esecuzione affidabile e scalabile per le applicazioni container.

4.1 Componenti Principali

4.1.1 Master Node

Il cuore del cluster Kubernetes è costituito da un set di nodi master che coordinano tutte le attività nel cluster. I principali componenti del nodo master sono:

- **kube-apiserver:** È il componente che espone l'API Kubernetes. Tutte le operazioni sui cluster, come la creazione, la modifica e la cancellazione di risorse, vengono effettuate attraverso questa API. È il punto di ingresso per i client e il componente principale per la comunicazione con il cluster.
- **kube-controller-manager:** Questo componente include diversi controller che monitorano continuamente lo stato del cluster e cercano di avvicinarlo allo stato desiderato. Ad esempio, il controller dei nodi

⁶Fonte: <https://omardiaa.hashnode.dev/kubernetes-101>.

si assicura che il numero di nodi nel cluster corrisponda alla configurazione desiderata, mentre il controller delle repliche gestisce i pod replicati per garantire la disponibilità e l'affidabilità delle applicazioni.

- **kube-scheduler:** È responsabile dell'assegnazione dei pod ai nodi del cluster. Utilizza diverse politiche di pianificazione, come i requisiti di risorse, la tolleranza ai guasti e la località dei dati, per selezionare il nodo migliore su cui eseguire un determinato pod.

4.1.2 Worker Node

Questi nodi eseguono il lavoro effettivo nel cluster Kubernetes, ospitando i container che compongono le applicazioni. I principali componenti del nodo worker includono:

- **kubelet:** È un agente che viene eseguito su ciascun nodo del cluster e gestisce i container su quel nodo. Riceve le specifiche dei pod dal kube-apiserver e si assicura che i container all'interno dei pod siano in esecuzione e in uno stato sano.
- **kube-proxy:** È responsabile della gestione delle regole di rete all'interno del cluster. Si occupa di instradare il traffico di rete ai servizi all'interno del cluster e gestisce la comunicazione tra i pod e il mondo esterno.
- **Container Runtime:** È il software che esegue i container sui nodi del cluster. Kubernetes supporta diversi runtime container, tra cui Docker, containerd, cri-o, e cri-docke.

4.1.3 Kubectl

Kubectl è il client da riga di comando ufficiale per interagire con i cluster Kubernetes. Consente agli utenti di eseguire una vasta gamma di operazioni di gestione e monitoraggio del cluster, facilitando il deployment, la configurazione e la gestione delle risorse.

Le principali funzionalità di kubectl includono:

- **Deployment delle Applicazioni:** Kubectl consente agli utenti di distribuire le proprie applicazioni su un cluster Kubernetes in modo rapido e semplice, utilizzando comandi intuitivi e flessibili. Gli utenti possono specificare le risorse desiderate, come pod, deployment e servizi, e kubectl si occupa di orchestrare il deployment in modo efficiente.

- **Monitoraggio dello Stato delle Risorse:** Kubectl fornisce agli utenti uno strumento potente per ispezionare lo stato delle risorse all'interno del cluster. Gli utenti possono visualizzare informazioni dettagliate su pod, servizi, deployment e altri oggetti di Kubernetes, consentendo loro di identificare eventuali problemi o anomalie e prendere le azioni necessarie per risolverli.
- **Gestione delle Configurazioni:** Kubectl permette agli utenti di gestire facilmente le configurazioni del cluster, compresi i file di configurazione kubeconfig e le variabili di ambiente. Gli utenti possono aggiungere, rimuovere o modificare le configurazioni del cluster utilizzando semplici comandi da riga di comando, semplificando il processo di gestione e manutenzione del cluster.
- **Esplorazione delle Risorse:** Kubectl offre agli utenti uno strumento intuitivo per esplorare e navigare attraverso le risorse all'interno del cluster Kubernetes. Gli utenti possono eseguire query complesse e filtrare le risorse in base a diversi criteri, consentendo loro di ottenere una visione dettagliata del cluster e delle sue componenti.
- **Interazione con l'API:** Kubectl consente agli utenti di interagire direttamente con l'API Kubernetes, consentendo loro di eseguire operazioni avanzate e personalizzate sul cluster. Gli utenti possono utilizzare kubectl per inviare richieste RESTful all'API Kubernetes e ottenere risposte dettagliate, aprendo nuove possibilità per l'automazione e l'integrazione del cluster con altri sistemi e strumenti.

4.1.4 Altri componenti

- **Pods:** I pod sono l'unità di base di distribuzione in Kubernetes. Un pod può contenere uno o più container che condividono risorse di rete e di archiviazione. Kubernetes gestisce i pod come una singola entità e li programma su nodi worker.

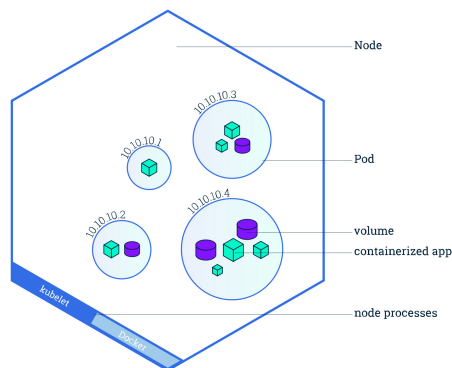


Figure 6: Overview di un generico pod kubernetes⁷

- **Servizi:** I servizi Kubernetes forniscono un'astrazione di rete stabile per i pod. Consentono ai pod di comunicare tra loro senza dover conoscere i dettagli della loro implementazione o della loro posizione.
- **Controller:** I controller Kubernetes monitorano lo stato del cluster e lavorano per portare il cluster nello stato desiderato. Questi includono controller per nodi, repliche, servizi e altri oggetti di Kubernetes.
- **Volume:** I volumi Kubernetes forniscono un meccanismo per persistere i dati oltre il ciclo di vita dei singoli pod. Possono essere montati nei container per archiviare dati in modo persistente.

La struttura di Kubernetes è progettata per garantire l'affidabilità, la scalabilità e la portabilità delle applicazioni containerizzate, consentendo agli sviluppatori di concentrarsi sulla logica dell'applicazione senza doversi preoccupare dei dettagli operativi sottostanti.

⁷Fonte: <https://kubernetes.io/docs/tutorials/kubernetes-basics/explore/explore-intro/>.

5 K3S Lightweight Kubernetes

K3s è una distribuzione leggera di Kubernetes progettata per fornire un'esperienza semplificata nella gestione dei container. Rispetto Kubernetes standard, K3s conserva le funzionalità principali, ma è ottimizzata per essere più veloce da installare, richiedendo meno risorse di sistema e occupando meno spazio su disco. Questa caratteristica lo rende particolarmente adatto per ambienti edge, dispositivi IoT e altri scenari in cui le risorse sono limitate. Inoltre, K3s semplifica il processo di configurazione e manutenzione, consentendo agli utenti di avviare rapidamente e facilmente cluster di container senza dover affrontare la complessità di una distribuzione standard di Kubernetes.

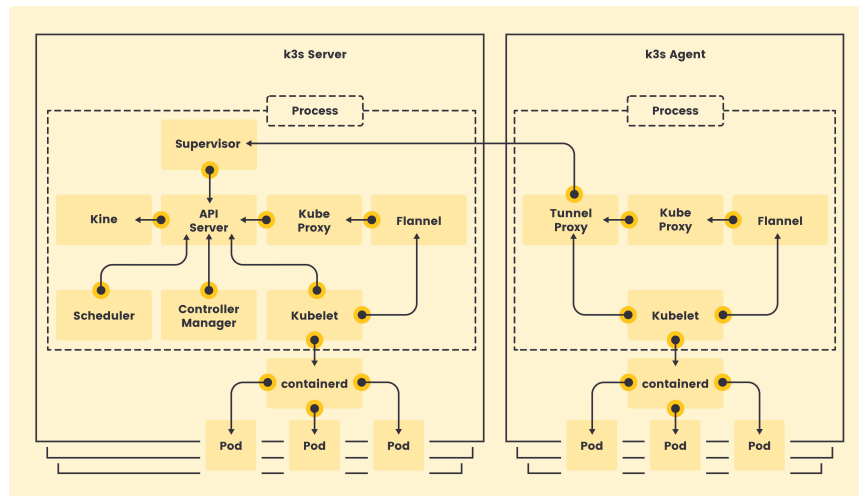


Figure 7: Generica architettura k3s ⁸

Le caratteristiche principali di k3s sono le seguenti:

- **Architettura leggera:** K3s semplifica l'architettura di Kubernetes, combinando i componenti del master e del worker in un unico processo. Ciò riduce il consumo di risorse e semplifica l'installazione e la manutenzione.
- **Deployment semplificato:** K3s può essere distribuito facilmente su diversi tipi di infrastrutture, inclusi dispositivi edge, server virtuali e

⁸Fonte: <https://k3s.io>

cloud. L'installazione è veloce e richiede poche risorse, il che lo rende adatto per ambienti con limiti di risorse.

- **All-in-One:** K3s include tutti i componenti necessari per eseguire un cluster Kubernetes completamente funzionale, compresi kube-apiserver, kube-controller-manager, kube-scheduler, kubelet, kube-proxy e un'implementazione del container runtime.
- **Sicurezza integrata:** K3s include funzionalità di sicurezza integrate, come il supporto per il trasporto sicuro dei dati tramite TLS e la possibilità di abilitare l'autenticazione e l'autorizzazione basate su ruoli.
- **Gestione semplificata:** K3s offre strumenti per semplificare la gestione e il monitoraggio del cluster, inclusi un'interfaccia API RESTful per l'automazione delle operazioni e un'ampia gamma di strumenti di monitoraggio e logging.

In k3s il nodo master viene chiamato nodo server, mentre il nodo worker prende il nome di nodo agent. La loro creazione e configurazione, nel caso d'uso specifico, verrà presentata nel prossimo capitolo.

5.1 Implementazione con K3s

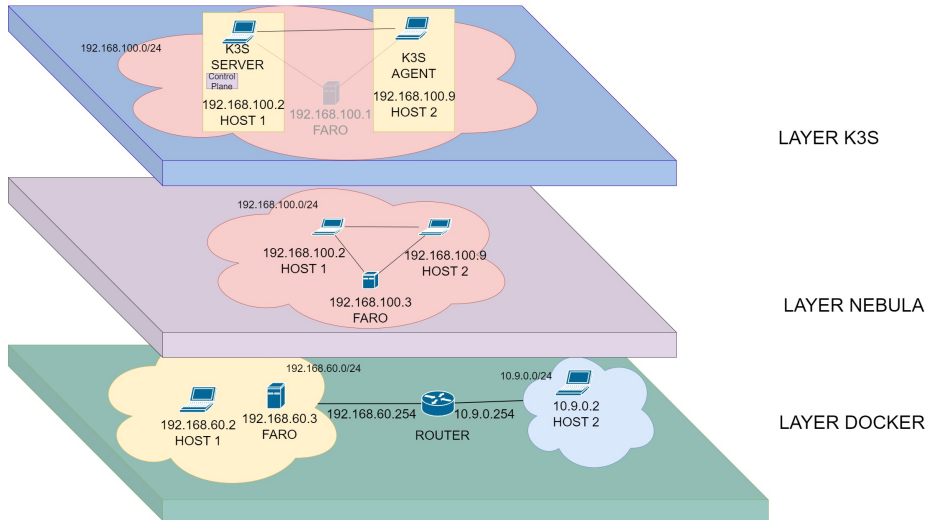


Figure 8: Architettura k3s

Come illustrato nella Figura 6, l'implementazione di k3s si basa, ovviamente, sulla rete overlay nebula. Viene identificato come server l'Host1 mentre come agent l'host2, l'utente può comunicare e gestire i cluster con l'utilità Kubectl. Nei sottoparagrafi successivi verrà descritto come è stato messo su il nodo server e il nodo agent.

5.1.1 K3s server

L'immagine base utilizzata, che fa parte del progetto k3s-dind [2] viene fornita con k3s disponibile, ed assieme, uno script che avvia un server. Il codice, del nodo server, interessante è il seguente:

```
function runDocker {
    dockerd \
    --host=unix:///var/run/docker.sock \
    --host=tcp://0.0.0.0:2375 \
    > /var/log/docker.log 2>&1 < /dev/null &

    until dockerReady ; do
```

```
        sleep 1
    done
}
#Viene avviato per prima il servizio docker, sul
#quale si appoggia K3s per far virtualizzare i
#container che dovra' orchestrare

K3S_NAME=$(hostname)
K3S_ARGS=( \
    --no-deploy=traefik \
    --docker \
    --https-listen-port=${K3S_API_PORT:-8443} \
    --node-name=${K3S_NAME} \
    --tls-san=${K3S_NAME} \
)

function runServer {
    k3s server "${K3S_ARGS[@]}" >> ${K3S_LOG} 2>&1 &
}

echo > ${K3S_LOG}
tail -F ${K3S_LOG} &

runDocker
runServer
```

il server K3s è responsabile di gestire il nodo master all'interno di un cluster Kubernetes. Gestisce le operazioni di orchestrazione, come la pianificazione dei container, il monitoraggio delle risorse e la gestione dello stato dell'applicazione

5.1.2 K3s agent

Il nodo K3s agent è responsabile di eseguire le computazioni all'interno del cluster Kubernetes. Questi nodi sono chiamati anche "worker nodes" o "nodi minion". Il compito principale di un nodo agente è quello di eseguire i container e le applicazioni assegnate loro dal nodo master. Ogni nodo agente contribuisce alle capacità computazionali complessive del cluster, lavorando insieme agli altri nodi per distribuire e gestire le applicazioni in modo efficiente e affidabile, viene creato con il seguente comando:


```
k3s agent --server https://ip-server:porta --token  
<NODE-TOKEN>
```

il flag **—server https://192.168.100.2:8442** indica l’ip e la porta del server, mentre il **—token** indica il token di accesso del server che serve per portare a termine l’autenticazione e autorizzare i nodi all’interno del cluster.

6 Conclusioni

In conclusione, la crescente necessità di tecnologie con bassa latenza e elevate capacità di calcolo ha messo in discussione l'efficacia del tradizionale modello basato sui datacenter. Questo modello, pur essendo stato per lungo tempo il punto di riferimento per l'elaborazione e la trasmissione dei dati, si è dimostrato spesso non ottimale, soprattutto in settori critici come la telemedicina e la guida autonoma, dove la latenza elevata può compromettere la sicurezza e l'efficacia del servizio.

L'adozione di un approccio distribuito e orientato ai dispositivi periferici si è rivelata una risposta promettente a queste sfide. Questo nuovo approccio consente un'elaborazione e una risposta più rapide, riducendo così la latenza e migliorando l'esperienza utente. I dispositivi periferici, distribuiti più vicino agli utenti finali, non solo raccolgono e trasmettono i dati, ma eseguono anche calcoli e decisioni localmente, offrendo una soluzione più efficiente e resiliente alle esigenze delle applicazioni sensibili alla latenza.

L'architettura proposta, con i suoi tre livelli di Docker, Nebula e K3s, rappresenta un'implementazione concreta di questo nuovo approccio. Docker virtualizza l'infrastruttura di rete, Nebula implementa un overlay network per agire come una sola rete logica riducendo le eventuali disomogeneità, mentre K3s gestisce i container Kubernetes per le computazioni, consentendo la distribuzione dinamica dei servizi e la gestione efficiente delle risorse.

Con l'infrastruttura basata su K3s disponibile e i nodi worker pronti, è possibile distribuire e far eseguire servizi e applicazioni su di essi. Kubernetes si occuperà quindi di distribuire e gestire tali risorse sui nodi worker disponibili, garantendo che le applicazioni siano eseguite in modo affidabile, scalabile e resilienti, abilitando inoltre la migrazione della computazione da un nodo ad un altro, funzione che è in fase di implementazione e di test.

References

- [1] Nebula. “Using Experimental Lighthouse DNS with Nebula”. In: *Nebula Docs* (). URL: <https://nebula.defined.net/docs/guides/using-lighthouse-dns/>.
- [2] unboundedsystems. “k3s-dind”. In: *Github* (). URL: <https://github.com/unboundedsystems/k3s-dind>.

Ringraziamenti

Finalmente siamo arrivati alla fine della prima tappa di questo viaggio.

Prima di tutti vorrei ringraziare i miei genitori, che mi hanno sempre sostenuto in qualsiasi cosa e permesso, grazie al loro sacrificio, di inseguire i miei sogni fin da bambino.

Grazie agli amici di una vita, Alessandro, Lorenzo, Luca, Simone, Francesco, Manuel, Gabriele, Daniele, e tutti quelli che non sono potuti esserci in questo giorno, per aver sempre creduto in me e nelle mie capacità, a volte anche più di me stesso.

Una parte importante, forse la più importante, di questo percorso è stata la compagnia, conoscere persone così straordinarie è stato l'ingrediente segreto che mi ha permesso di arrivare fino alla fine.

Grazie agli amici dell'Università, Albi, Mic, Samu, Boris, Dennis e Eusebio per aver alleggerito le lezioni e per gli innumerevoli suggerimenti.

Grazie agli amici del gruppo "Cilenzio", Alessio, Biagio, Carlotta, Donato, Gabriele, Gianmarco, Giuliano, Leo, Lorenzo, Luca, Maddalena, Margherita, Mario, Martina, Sandro e Simone e Arianna per le infinite serate alle Panche, per i pomeriggi passati al Valentino, per le nottate all' Audiodrome e tanto tanto altro. Ogni risata, ogni abbraccio e ogni momento di condivisione hanno reso questo viaggio un'esperienza unica e indimenticabile. Stare con voi ha addolcito la nostalgia di casa. Sono sicuro che senza di voi, Torino, per me, non avrebbe lo stesso sapore.

Anche per questo un ringraziamento speciale va ad Antonio, che ha funto da anello di congiunzione tra voi e me e ad Alex, che insieme a lui è stato un faro nei primi mesi del mio arrivo.

Cos'è più importante - chiese grande panda -: il viaggio o la meta?"

"La compagnia", risponde piccolo drago.

E così, con le parole di James Norbury nel suo libro 'Grande Panda e Piccolo Drago', concludo questo capitolo del mio percorso accademico.

A tutti quelli che hanno condiviso con me questo cammino Grazie.

Vi voglio bene.

Salvatore.